EP08 Pattern Recognition – Machine Learning

Assignment 1: Classical Classification Algorithms on Real-World Data

Assignment type: ***Individual***

Submission deadline: ***Sunday 03/05/2026, 23:55*** (***No extensions will be granted***)

Submission method: ***Exclusively via eclass*** (***assignments sent by email will not be accepted***)

Total points: ***100*** (***40 points Notebook + 60 points oral examination***)

(20% of the final course grade)

The oral examination will take place shortly after the submission deadline.

Objective

In this assignment you will choose one of three tasks (deepfake detection, environmental sound classification, or human activity recognition), train five classical algorithms on a public dataset, and evaluate how well they generalize to data that you collect yourselves.

Specifically, you will develop the full cycle of a supervised learning system: data analysis, feature extraction, training and comparison of classical classification algorithms (k-NN, Naive Bayes, Logistic Regression, SVM, Decision Tree), as well as evaluation of generalization on your own data.

The notebook is your preparation; the main assessment takes place in the oral examination, where you will be questioned on your code and your results as submitted on eclass. If the results (models, plots, metrics) are not available, or you cannot explain your choices and your results, the grade for the assignment will be correspondingly low, regardless of the quality of the code.

Answer structure

The assignment consists of 18 questions (Q0.1–Q3.7) that are answered inside the notebook, in a markdown cell immediately after the corresponding code, with a title exactly the same as in the assignment text (e.g. ## Q0.1 If the dataset has class imbalance...). Each answer must be short (5–10 sentences), answer the question directly, and be supported by reference to specific results of the notebook (metrics, plots, tables).

An answer that remains at a general theoretical level, without reference to the results of your notebook, is considered incomplete.

Task Selection

Choose one task from the list below.

	Task	Kaggle Dataset	How to collect your own data
A	Deepfake Detection	FaceForensics++ subset	Take 15–20 photos (selfies, different lighting/ background). Create the deepfake version of each photo using a face swap tool of your choice (e.g. FaceSwapper.ai, Reface, FaceSwap or similar). Save as png or jpg. Record which tool you used in a CSV (columns: filename, label, tool), since the generation method affects the artifacts you will try to detect. Result: 15–20 pairs (genuine + synthetic image).
B	Sound Classification	UrbanSound8K (Kaggle)	Record 20–30 environmental sounds with your phone (Voice Memo, RecForge, or any recording app), 3–5 seconds long each. Choose at least 3 of the UrbanSound8K categories and split the recordings evenly among them. Save as wav or m4a. Record the category of each recording in a CSV (columns: filename, category). Result: 20–30 recordings in at least 3 categories.
C	Human Activity Recognition	UCI HAR (Kaggle)	Install a sensor-logging app on your phone (e.g. Phyphox, Physics Toolbox Suite, or Sensor Logger). Record 20–30 motion samples, 5–10 seconds long each. Choose at least 3 of the UCI HAR categories (e.g. walking, running, climbing stairs, sitting, standing) and split the samples evenly among them. Save as CSV. Record the activity of each sample in a CSV (columns: filename, activity). Result: 20–30

			accelerometer recordings in at least 3 categories.

0. Data

0.1 Kaggle dataset

Here you load and prepare the Kaggle dataset. It is the basis of all the experiments that follow.

If the Kaggle dataset you chose already contains pre-computed features, use its raw version (images, recordings, keystroke logs). Feature extraction is your own responsibility and is assessed in Section 1.

- Load the dataset. Print: total size, number of classes, number of samples per class.
- Split into 70% training, 15% validation, 15% test, with stratify=y. Print how many observations per class went into each set.

Q0.1 (2 pts)

If the dataset has class imbalance, which specific evaluation metric becomes misleading and why? What problem does the imbalance cause in training? What did you do about it and why?

0.2 Own data

Here you collect and document your own data. You will use them in Section 3.

- Collect data samples according to the task instructions.
- Manually label the category of each sample. Explicitly record the criteria you followed for the annotation.
- Show basic statistics: how many samples per class, under which conditions they were collected.

Q0.2 (1 pt)

What biases might the way you collected the data introduce? Is there a homogeneity that does not reflect the diversity of the real population?

1. Feature Extraction

1.1 Feature extraction

First, extract the features that will be fed to the algorithms. The rationale for each task is given below. Read the corresponding paragraph and implement the basic features. Then add at least one extra feature, either from the Optional ones or of your own design, and justify why you chose it (see Q1.1).

Task A: Deepfake Detection

A genuine photo has consistent statistical characteristics across the whole image: the lens, the sensor and the lighting affect all pixels in the same way. A deepfake replaces or modifies part of the image (usually the face), with the result that the statistical characteristics of the edited region deviate from those of the rest of the frame. These deviations are not always visible to the naked eye, but they can be detected numerically.

Load each image and convert it to RGB. Resize to a fixed resolution (e.g. 256×256) so that all feature vectors have the same size.

Extract the following features:

- **Per-channel RGB statistics**: np.mean(img_rgb[:,:,i]) and np.std(img_rgb[:,:,i]) for i = 0, 1, 2 (6 features). In deepfakes, the relative brightness between channels tends to change because of a different color pipeline in the edited region.
- **Color histogram**: np.histogram(channel.flatten(), bins=32, density=True) for each channel (3 × 32 = 96 features). Deepfakes often produce a more uniform intensity distribution compared to natural photos.
- **Image entropy**: scipy.stats.entropy(hist + 1e-10) per channel (3 features). Entropy measures how varied the pixels are. Low entropy means a homogeneous image, high entropy means great variety. Edited regions tend to have different entropy from genuine ones.
- **Local statistics (patch-based)**: split the image into 4×4 patches (16 regions) and compute mean and std per channel per patch (16 × 3 × 2 = 96 features). Global statistics may fail to capture local anomalies; if the edited region (e.g. the face) has different statistics from the rest of the background, the variance between patches will be unusually high. Also compute the standard deviation between patches (np.std over the 16 patch means per channel, 3 features) as a measure of local inconsistency.

Optional: **FFT analysis**: magnitude = np.abs(np.fft.fft2(channel)) extracts features from the frequency domain (mean, std, max magnitude). Deepfakes introduce characteristic artifacts in the high frequencies due to upsampling and blending. **Noise residual analysis**: apply a high-pass filter to each channel (e.g. residual = img - cv2.GaussianBlur(img, (5,5), 0)) and compute the mean, std, skewness of the residual per channel (9 features). In a genuine photo, the sensor noise is uniform across the whole image; in deepfakes, the edited region has gone through a different pipeline (generation, blending, compression), so the noise statistics differ from those of the rest of the frame.

Task B: Sound Classification

Sound is a waveform with thousands of values per second. We cannot feed them to the classifier as they are; we need fixed-size summary representations that describe the character of each sound regardless of its duration. Trim or pad each clip to a fixed duration (e.g. 4 seconds) so that all feature vectors have the same size.

Extract the following features:

- **MFCC** (Mel-Frequency Cepstral Coefficients): librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13). They represent the spectral envelope, i.e. the timbre of the sound, which is what distinguishes e.g. a car horn from a dog bark even if they have similar loudness. For each clip: np.mean(mfcc, axis=1) and np.std(mfcc, axis=1) (13 + 13 = 26 features).
- **Spectral centroid**: np.mean(librosa.feature.spectral_centroid(y=y, sr=sr)) (1 feature). It represents the center of mass of the spectrum and is high for high-pitched sounds (e.g. whistling) and low for low-pitched ones (e.g. an engine).
- **Zero crossing rate**: np.mean(librosa.feature.zero_crossing_rate(y)) (1 feature). It expresses how often the signal changes sign and is high for noisy sounds (rain, friction) and low for tonal ones (music, voice).

Optional: **Spectral contrast**: librosa.feature.spectral_contrast(y=y, sr=sr) measures the energy difference between peaks and valleys in the spectrum. **Chroma features**: librosa.feature.chroma_stft(y=y, sr=sr), which represents the energy distribution over the 12 musical notes. For both: mean and std per clip.

Task C: Human Activity Recognition

Accelerometer data are three-axis time series (x, y, z) with dozens of values per second. Each activity has a characteristic "signature": walking produces a rhythmic alternation, running a larger amplitude, standing an almost constant signal.

Extract the following features:

- **Per-axis statistics**: mean, std, min, max for each axis x, y, z (12 features). The mean and standard deviation of each axis capture the overall position and the intensity of the motion.
- **Magnitude**: mag = np.sqrt($x^2 + y^2 + z^2$), then mean and std (2 features). The intensity of motion independent of the phone's orientation; it separates stationary from dynamic activities.
- **Zero crossing rate**: how often each axis crosses its mean value (3 features). High for rhythmic movements (walking, running), low for stationary ones.

Optional: **FFT dominant frequency**: np.fft.fft(axis); the dominant frequency separates walking (~2 Hz) from running (~3 Hz). **Jerk signal**: np.diff(axis), the derivative of acceleration, sensitive to abrupt changes in motion.

	Library	Basic features	Optional
A	opencv-python, scipy	Per-channel RGB statistics (mean, std), 32-bin color histogram, image entropy, patch-based	FFT-based features, etc
B	librosa	MFCC 13 (mean + std per clip), spectral centroid (mean), zero crossing rate (mean)	Spectral contrast, chroma features (mean + std per clip)

C	pandas, numpy, scipy	Per-axis statistics (mean, std, min, max), magnitude (mean, std), zero crossing rate	FFT dominant frequency, spectral energy, jerk signal statistics

Q1.1 (1 pt)

Which extra features did you implement (at least one)? Which hypothesis of yours led to choosing them? Based on the histograms and the correlation matrix (§1.3), do they seem to differentiate the classes?

1.2 Feature preprocessing

Now that you have extracted the features, apply the preprocessing steps required before training.

- Missing values: document whether there are any and what you did.
- Normalization of numerical features: choose a method (e.g. StandardScaler, MinMaxScaler, etc.), fit only on the training set, transform the validation and test sets. Justify your choice.

Q1.2 (2 pts)

How did you handle missing values (if any)? Justify how you chose the normalization method.

1.3 Exploration

Explore the structure of the features before starting any modeling.

- Distribution histograms of each feature, separately per class.
- Correlation matrix (heatmap). Identify pairs of highly correlated features; these are candidates for removal.

Q1.3 (2 pts)

Which features differ between classes and which do not? If many features are correlated with each other, which classification algorithms will be affected the most and why?

1.4 Dimensionality reduction and visualization

Visualization in low dimensions reveals whether the classes are linearly separable and informs the choice of classification algorithm (and kernel).

- PCA in 2 dimensions: scatter plot color-coded by class. Report the explained variance ratio.
- t-SNE and UMAP: compare the result with that of PCA.

Q1.4 (2 pts)

To what extent are the classes separated in the visualization? Based on it, which algorithms are expected to perform well and which not, and why?

2. Training and Comparison of Algorithms

Implement and evaluate the following algorithms with scikit-learn: k-NN, Naive Bayes, Logistic Regression, SVM, Decision Tree. The evaluation workflow is: (1) 5-fold CV on the training set to select hyperparameters for each algorithm, (2) training with the best hyperparameters on the whole training set and evaluation on the validation set to compare the algorithms, (3) final evaluation of the best model on the test set.

Before training any algorithm, run a DummyClassifier(strategy='most_frequent') and record its performance. This sets the minimum acceptable threshold. Any model that does not beat it offers no meaningful learning.

Note: at every point where randomness is involved (split, cross-validation, training) use random_state=42 so that the results are reproducible during the oral examination.

The "Comprehension question" column is not graded in the notebook, but serves as preparation for the oral examination.

Algorithm	Hyperparameters for cross-validation	Comprehension question
k-NN	k ∈ {1, 3, 5, 11, 21}, metric ∈ {euclidean, cosine}	What does 'distance' mean in the feature space of your task?
Naive Bayes	GaussianNB, BernoulliNB (if the features are binary)	Is the independence assumption violated in your data?
Logistic Regression	C ∈ {0.01, 0.1, 1, 10, 100}, penalty ∈ {l1, l2} (use solver='saga' or 'liblinear' for l1)	What happens for a very small C? For a very large one?
SVM	kernel ∈ {linear, rbf, poly}, C ∈ {0.1, 1, 10, 100}	Which kernel is suggested by the t-SNE visualization?
Decision Tree	max_depth ∈ {3, 5, 10, None}, min_samples_leaf ∈ {1, 5, 10}	How does max_depth control overfitting?

Q2.1 (3 pts)

Fill in the table:

Algorithm	Best hyperparameters	Val Acc (%)	Test Acc (%)	F1 macro (%)
k-NN				
Naive Bayes				
Logistic Regression				
SVM				
Decision Tree				
Naive Baseline				

Q2.2 (2 pts)

Show the confusion matrix for the best algorithm. Thinking about the features, which classes are confused with each other and why?

Q2.3 (2 pts)

Find at least three observations (data samples) that all algorithms misclassify. Are the features to blame, or are the data themselves ambiguous?

Q2.4 (3 pts)

Scenario: Your team lead at a company asks: "Why don't we use the algorithm with the best accuracy everywhere?"

Based on your results, when is a simpler algorithm the better choice? Consider performance, interpretability, computational cost.

3. Evaluating the Classifiers under Distribution Shift

In this section you empirically examine what happens when a model is applied to data with a different distribution from the one it was trained on (distribution shift). It is one of the central problems of ML in real-world conditions.

3.0 Feature extraction from your own data

Before evaluating the model, you must extract the same features from the data you collected in Section 0.2. Use exactly the same extraction code you applied to the Kaggle dataset; a uniform procedure is necessary for a valid comparison.

- Apply the same feature extraction code (Task A/B/C) to your own data.
- Apply the same normalization transform: use the scaler fitted on the Kaggle training set, without a new fit.
- Make sure the feature vector has the same size as the Kaggle one.

Q3.0 (1 pt)

Did you face any problem when extracting features from your own data, e.g. different audio duration, different image format, missing values? How did you deal with it?

3.1 First evaluation on your own data

Test your best model, as is, without retraining, on the data you collected.

- Apply the best model (based on Q2.1) to your own data.
- Compute accuracy, precision, recall and F1 per class.
- Confusion matrix and identification of classes with a high error rate.

Q3.1 (2 pts)

How much does performance drop compared to the Kaggle test set? Which class is affected the most? Does this seem reasonable to you?

Q3.2 (2 pts)

Scenario: A fraud detection system, developed for the Western European market, loses 22 points of accuracy when deployed in a new market.

What could the causes be? What would you check before every new deployment?

3.2 Distribution shift analysis

Identify which features differ between the Kaggle dataset and your own data, and whether this difference explains the errors.

- For each feature: histogram of Kaggle and own data on the same plot.

- Compute |mean_kaggle − mean_own| / std_kaggle per feature. Sort in descending order.
- Link the high-deviation features to the error categories in the confusion matrix.

Q3.3 (4 pts)

Which features deviate the most between Kaggle and your own data? Which external factors (equipment, environment, recording conditions) explain this deviation? Fill in the table with the 5 features with the largest deviation.

Feature Name	Mean (Kaggle)	Mean (Own)	Remarks

Q3.4 (2 pts)

Are the features with the largest deviation also responsible for most errors? Which one would you replace first?

3.3 Improvement strategies

Now that you know where and why the model fails, try at least two interventions.

- Intervention 1: split your own data into 50% own-train and 50% own-test. Use the best hyperparameters from §2. (a) Add own-train to the Kaggle training set and retrain. (b) Train only on own-train, without Kaggle. Record the performance of both versions on own-test and on the Kaggle test set. Comment on the difference — especially if version (a) does not noticeably improve performance, explain why. Note: version (b) is expected to perform poorly due to very little training data; what is assessed is the interpretation of the result, not the performance itself.

- Intervention 2: remove the feature with the largest deviation (based on §3.2) and retrain. Record the performance both on your own data and on the Kaggle test set.

Q3.5 (4 pts)

Fill in the table:

Evaluation scenario	Accuracy (%)	F1 macro (%)	Remarks
Kaggle test set (§2)			
Own data (without retraining)			
Own data with Intervention 1 (retraining)			
Own data with Intervention 2			

Q3.6 (2 pts)

Which intervention worked best? Why do you think it helped (or did not help)?

Q3.7 (3 pts)

If you could redesign the collection process of your own data from scratch, what would you change to reduce the distribution shift?

Oral Examination (60 points)

Duration: 10 minutes per student. You will be questioned on the notebook and your results. You will be asked to explain your choices and to answer new questions.

The oral examination is based on the notebook as submitted. If the results (models, plots, metrics) are not available or cannot be presented immediately, the assignment is considered insufficiently documented.

Oral examination grading scale: 55–60: in-depth understanding, ability to answer new questions, 40–54: adequate understanding, justification of basic choices, 20–39: superficial knowledge, difficulty with follow-up questions, 0–19: inability to explain

Grading

Section	Points
0. Data	3
1. Feature Extraction	7
2. Training and Comparison of Algorithms	10
3. Evaluating the Classifiers under Distribution Shift	20
Notebook Total	**40**
Oral Examination	**60**
Total	**100**

In case of technical problems

If you encounter a serious technical problem:

- Write in the notebook what you tried, what error you got, and what you did to solve it.
- If an algorithm produces an error, explain what you think is wrong and what you would expect if it worked.

– A technical problem that is properly documented does not imply a grade reduction, provided the rest of the assignment is complete.

Use of AI Tools

You may use AI tools (ChatGPT, Claude, Copilot, etc.) for debugging, syntax questions and clarifying concepts. You are not allowed to ask an AI to write the answers to the questions or to interpret results on your behalf. In the oral examination you may be asked to explain parts of the code that appear to have been generated by an AI tool.

You must include an AI_declaration.md file in which you state which tools you used, for what purpose, and declare that you understand the code and the results.

Implementation and submission instructions

The assignment is implemented in a Jupyter notebook (Google Colab or a similar environment). Make sure the notebook runs without errors from start to finish (Run All) before submission.

Each question is answered in a markdown cell immediately after the corresponding code, with clear formatting and structure to make reading and grading easier.

For reproducibility, add at the beginning of the notebook:

```
import numpy as np

import random

np.random.seed(42)

random.seed(42)
```

In all models use random_state=42 so that the results are reproducible during the oral examination.

ZIP file named **AM_Surname_TaskX.zip**

(e.g. 1115XXXX_Papadopoulos_TaskA.zip), containing:

- **AM_Surname_TaskX.ipynb:** notebook with code, results, and answers.
- **data/** the data you collected, screenshots of the collection process, and your labels.
- **README.txt** full name, AM (student ID), task, list of dependencies (pip install ...), estimated running time.
- **AI_declaration.md**

Do not include the Kaggle dataset; the code that downloads it is enough.